

Performance Testing

Date	14 July 2026
Team ID	
Project Name	HDI Predictor (Human Development Index Predictor)
Team Size	1 (Individual Project)
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Python `requests` library (custom sequential load script)
Type of Testing	Response-time / load testing (sequential requests, single client)
Target Module / API	Home page, Predict page, /api/predict, /api/history
Test Environment	Local (Flask development server, pre-deployment baseline)
Test Date	14 July 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	Requests	Notes	Expected Outcome
1	Load the Home dashboard (stats + EDA images) repeatedly	10	Sequential	Consistently fast after first request (page is static HTML)
2	Load the Predict form page repeatedly	10	Sequential	Fast — no model inference on page load, form only
3	Submit /api/predict with varying indicator values	20	Sequential	Each call loads the cached model and returns a prediction
4	Fetch /api/history?limit=50 repeatedly	10	Sequential	Simple SQLite SELECT with an index on created_at

Step 3: Performance Test Results (measured)

S.No	Endpoint	Avg Response Time	Max Response Time	Target (<2s avg)	Status
1	GET /	59.0 ms	566.4 ms (first-hit, cold)	< 2000 ms	Pass
2	GET /predict	5.7 ms	12.1 ms	< 2000 ms	Pass
3	POST /api/predict	114.0 ms	2087.2 ms (first-hit, model load)	< 2000 ms avg	Pass
4	GET /api/history?limit=50	2.8 ms	4.0 ms	< 2000 ms	Pass

Observation: the single slow outlier (2087 ms) on `/api/predict` occurs only on the very first call, when the trained model is loaded from disk into memory for the first time. `model_utils.get_model()` caches it afterward (module-level singleton), so every subsequent call in the same run completed in under 15 ms. This matches the 16.7 ms best-case time seen in the same run. On the live Render deployment, the free-tier instance sleeps after inactivity, so the very first request after a period of no traffic will show a similar one-time cold-start delay (10–30 seconds) — documented in the project README so evaluators aren't surprised by it.

Conclusion

All four endpoints stayed well under the 2-second target once warmed up, confirming the app meets NFR-4 (Performance) from the Requirement Analysis phase.